

2

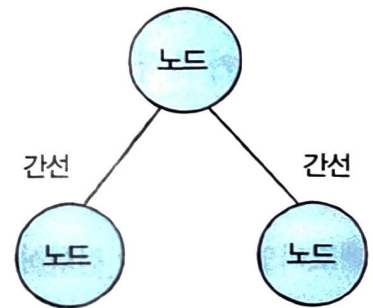
탐색 알고리즘 DFS/BFS

스택과 큐, 재귀 함수는 DFS와 BFS에서 가장 중요한 개념이라 DFS/BFS를 배우기에 앞서 간단하게 설명했다. 이제부터 DFS/BFS 알고리즘을 살펴보겠다.

DFS

DFS는 Depth-First Search, **깊이 우선 탐색**이라고도 부르며, 그래프에서 깊은 부분을 우선적으로 탐색하는 알고리즘이다. DFS를 설명하기 전에 먼저 그래프Graph의 기본 구조를 알아야 한다.

그래프는 **노드Node**와 **간선Edge**으로 표현되며 이때 노드를 **정점Vertex**이라고도 말한다. 그래프 탐색이란 하나의 노드를 시작으로 다수의 노드를 방문하는 것을 말한다. 또한 두 노드가 간선으로 연결되어 있다면 '두 노드는 인접하다Adjacent'라고 표현한다.

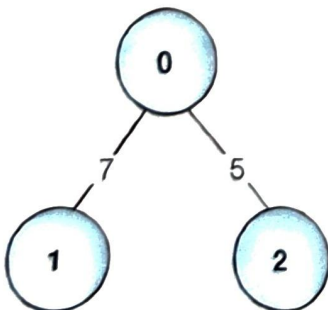


여기에서 갑자기 노드와 간선이라는 생소한 단어가 나와서 헷갈릴 수도 있는데, 일반적으로 그래프를 표현할 때 사용하는 단어들이다. 노

드를 도시, 간선을 도로라고 생각해보자. A라는 도시(노드)에서 B라는 도시(노드)로 이동하기 위해서, A와 B를 연결하는 도로(간선)를 거친다고 이해하면 쉬울 것이다.

프로그래밍에서 그래프는 크게 2가지 방식으로 표현할 수 있는데 코딩 테스트에서는 이 두 방식 모두 필요하니 두 개념에 대해 바르게 알고 있도록 하자.

- 인접 행렬(Adjacency Matrix): 2차원 배열로 그래프의 연결 관계를 표현하는 방식
- 인접 리스트(Adjacency List): 리스트로 그래프의 연결 관계를 표현하는 방식



	0	1	2
0	0	7	5
1	7	0	무한
2	5	무한	0

먼저 **인접 행렬Adjacency Matrix 방식**은 2차원 배열에 각 노드가 연결된 형태를 기록하는 방식이다*. 위와 같이 연결된 그래프를 인접 행렬로 표현할 때 파이썬에서는 2차원 리스트로 구현할 수 있다.

연결이 되어 있지 않은 노드끼리는 무한Infinity의 비용이라고 작성한다. 실제 코드에서는 논리적으로 정답이 될 수 없는 큰 값 중에서 999999999, 987654321 등의 값으로 초기화하는 경우가 많다. 이렇게 그래프를 인접 행렬 방식으로 처리할 때는 다음과 같이 데이터를 초기화한다.

5-6.py 인접 행렬 방식 예제

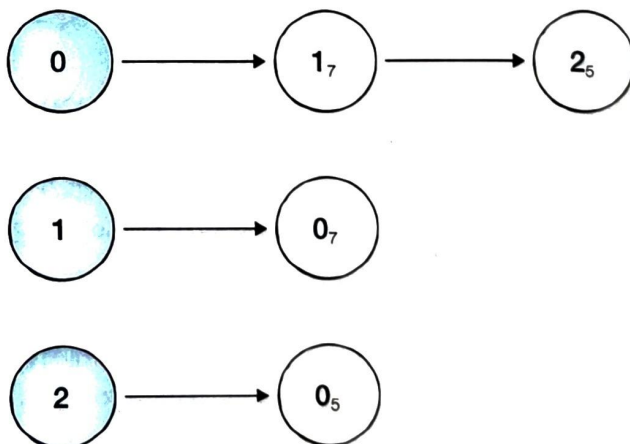
```
INF = 999999999 # 무한의 비용 선언

# 2차원 리스트를 이용해 인접 행렬 표현
graph = [
    [0, 7, 5],
    [7, 0, INF],
    [5, INF, 0]
]

print(graph)
```

```
[[0, 7, 5], [7, 0, 999999999], [5, 999999999, 0]]
```

그렇다면 **인접 리스트Adjacency List 방식**에서는 데이터를 어떤 방식으로 저장할까? 인접 리스트 방식에서는 다음 그림처럼 모든 노드에 연결된 노드에 대한 정보를 차례대로 연결하여 저장한다.



* 다른 언어의 배열(Array)을 파이썬에서는 리스트 자료형으로 표현할 수 있으므로 파이썬은 인접 행렬을 리스트로 구현한다.

인접 리스트는 '연결 리스트'라는 자료구조를 이용해 구현하는데, C++나 자바와 같은 프로그래밍 언어에서는 별도로 연결 리스트 기능을 위한 표준 라이브러리를 제공한다. 반면에 파이썬은 기본 자료형인 리스트 자료형이 `append()`와 메소드를 제공하므로, 전통적인 프로그래밍 언어에서의 배열과 연결 리스트의 기능을 모두 기본으로 제공한다. 파이썬으로 인접 리스트를 이용해 그래프를 표현하고자 할 때에도 단순히 2차원 리스트를 이용하면 된다는 점만 기억하자.

다음은 예제 그래프를 인접 리스트 방식으로 처리할 때 데이터를 초기화한 코드이다.

5-7.py 인접 리스트 방식 예제

```
# 행(Row)이 3개인 2차원 리스트로 인접 리스트 표현
graph = [[] for _ in range(3)]

# 노드 0에 연결된 노드 정보 저장(노드, 거리)
graph[0].append((1, 7))
graph[0].append((2, 5))

# 노드 1에 연결된 노드 정보 저장(노드, 거리)
graph[1].append((0, 7))

# 노드 2에 연결된 노드 정보 저장(노드, 거리)
graph[2].append((0, 5))

print(graph)
```

```
[[ (1, 7), (2, 5) ], [ (0, 7) ], [ (0, 5) ]]
```

이 두 방식은 어떤 차이가 있을까? 코딩 테스트를 위해 학습하는 터라 메모리와 속도 측면에서 살펴 보겠다. 메모리 측면에서 보자면 인접 행렬 방식은 모든 관계를 저장하므로 노드 개수가 많을수록 메모리가 불필요하게 낭비된다. 반면에 인접 리스트 방식은 연결된 정보만을 저장하기 때문에 메모리를 효율적으로 사용한다. 하지만 이와 같은 속성 때문에 인접 리스트 방식은 인접 행렬 방식에 비해 특정한 두 노드가 연결되어 있는지에 대한 정보를 얻는 속도가 느리다. 인접 리스트 방식에서는 연결된 데이터를 하나씩 확인해야 하기 때문이다.

또 다른 예시로 한 그래프에서 노드 1과 노드 7이 연결되어 있는 상황을 생각해보자. 인접 행렬 방식에서는 `graph[1][7]`만 확인하면 된다. 반면에 인접 리스트 방식에서는 노드 1에 대한 인접 리스

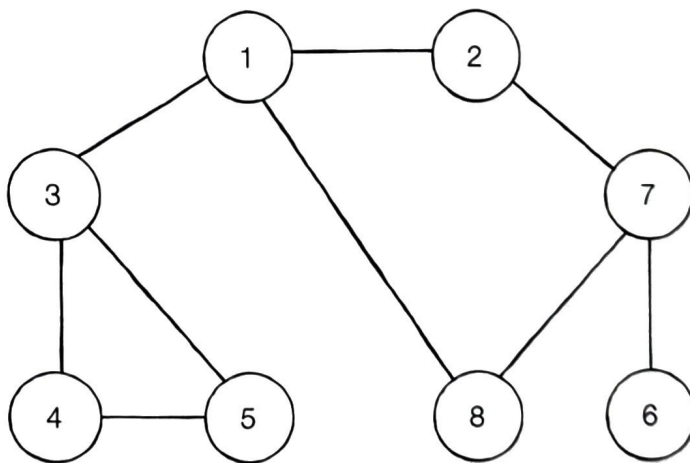
트를 앞에서부터 차례대로 확인해야 한다. 그러므로 특정한 노드와 연결된 모든 인접 노드를 순회해야 하는 경우, 인접 리스트 방식이 인접 행렬 방식에 비해 메모리 공간의 낭비가 적다. DFS는 탐색을 위해서 사용되는 탐색 알고리즘이라고 했는데 구체적으로 어떻게 동작할까? DFS는 깊이 우선 탐색 알고리즘이라고 했다. 이 알고리즘은 특정한 경로로 탐색하다가 특정한 상황에서 최대한 깊숙이 들어가서 노드를 방문한 후, 다시 돌아가 다른 경로로 탐색하는 알고리즘이다.

DFS는 스택 자료구조를 이용하며 구체적인 동작 과정은 다음과 같다.

- 탐색 시작 노드를 스택에 삽입하고 방문 처리를 한다.
- 스택의 최상단 노드에 방문하지 않은 인접 노드가 있으면 그 인접 노드를 스택에 넣고 방문 처리를 한다. 방문하지 않은 인접 노드가 없으면 스택에서 최상단 노드를 꺼낸다.
- 2번의 과정을 더 이상 수행할 수 없을 때까지 반복한다.

TIP '방문 처리'는 스택에 한 번 삽입되어 처리된 노드가 다시 삽입되지 않게 체크하는 것을 의미한다. 방문 처리를 함으로써 각 노드를 한 번씩만 처리할 수 있다.

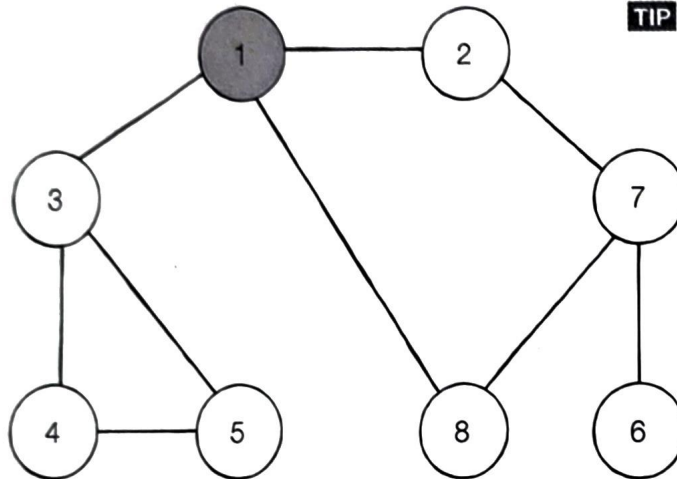
다음과 같은 그래프를 생각해보자. 노드 1을 시작 노드로 설정하여 DFS를 이용해 탐색을 진행하면 어떻게 될까? 직관적으로 생각하면, 깊이 우선 탐색이라는 이름에서부터 알 수 있듯이 단순히 가장 깊숙이 위치하는 노드에 닿을 때까지 확인(탐색)하면 된다.



DFS를 이용하여 탐색하면 그 과정은 다음과 같다. 또한 일반적으로 인접한 노드 중에서 방문하지 않은 노드가 여러 개 있으면 번호가 낮은 순서부터 처리한다.

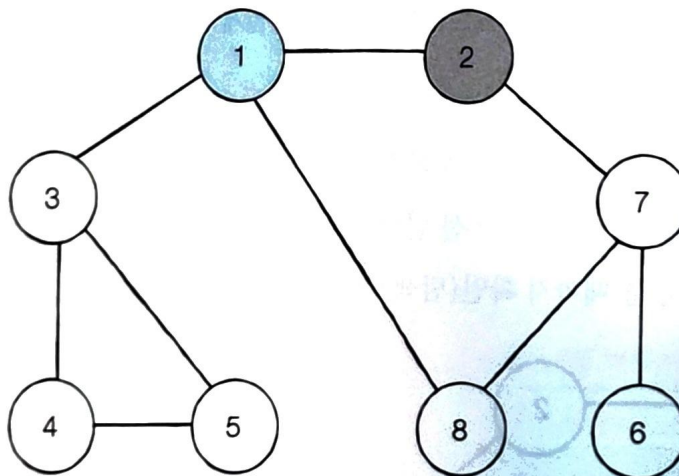
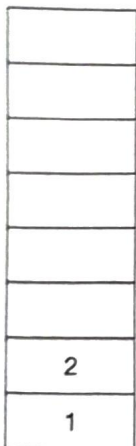
TIP DFS의 기능을 생각하면 순서와 상관없이 처리해도 되지만, 코딩 테스트에서는 번호가 낮은 순서부터 처리하도록 명시하는 경우가 종종 있다. 따라서 관행적으로 번호가 낮은 순서부터 처리하도록 구현하는 편이다.

step 1 시작 노드인 '1'을 스택에 삽입하고 방문 처리를 한다.

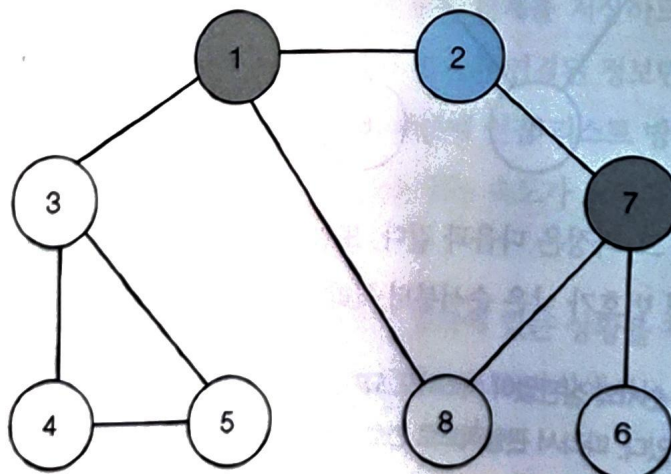
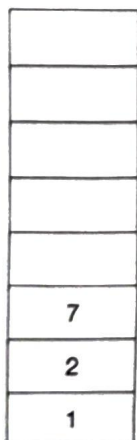


TIP 방문 처리된 노드는 회색으로, 현재 처리하는 스택의 최상단 노드는 하늘색으로 표현했다.

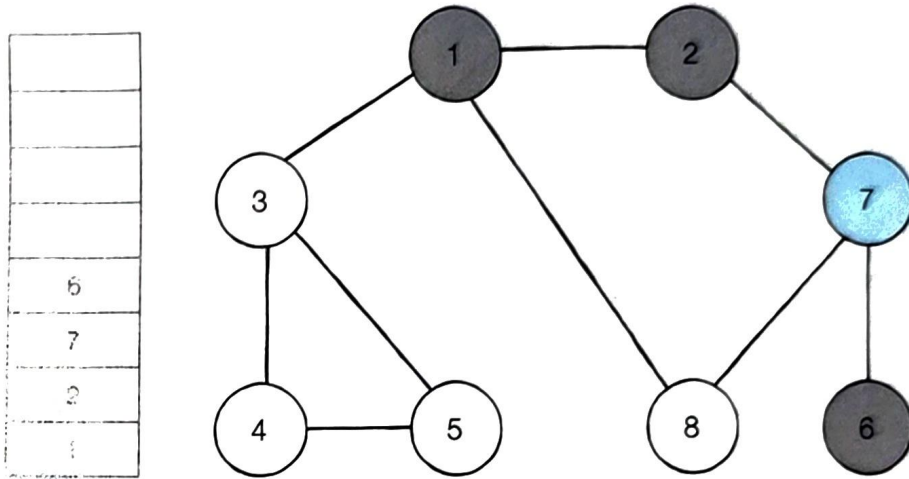
step 2 스택의 최상단 노드인 '1'에 방문하지 않은 인접 노드 '2', '3', '8'이 있다. 이 중에서 가장 작은 노드인 '2'를 스택에 넣고 방문 처리를 한다.



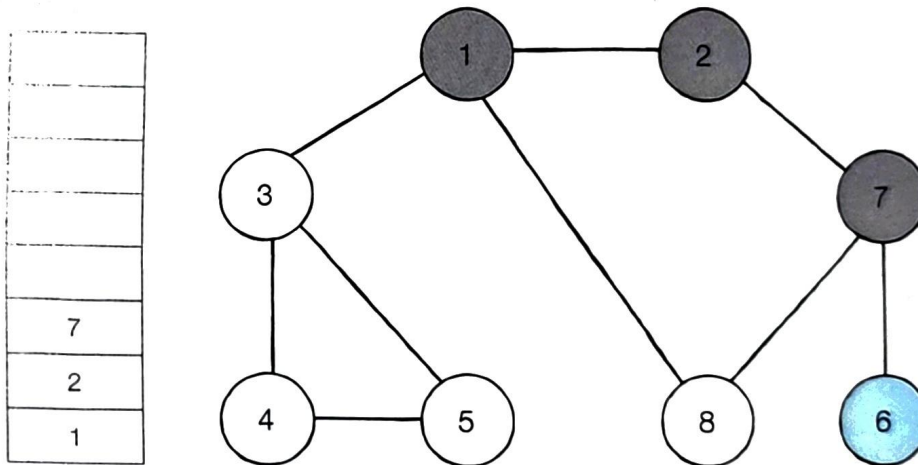
step 3 스택의 최상단 노드인 '2'에 방문하지 않은 인접 노드 '7'이 있다. 따라서 '7'번 노드를 스택에 넣고 방문 처리를 한다.



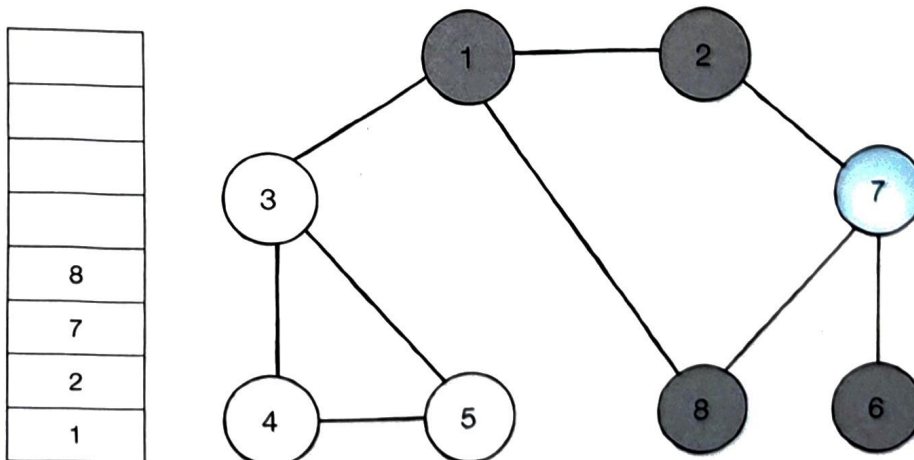
step 4 스택의 최상단 노드인 '7'에 방문하지 않은 인접 노드 '6'과 '8'이 있다. 이 중에서 가장 작은 노드인 '6'을 스택에 넣고 방문 처리를 한다.



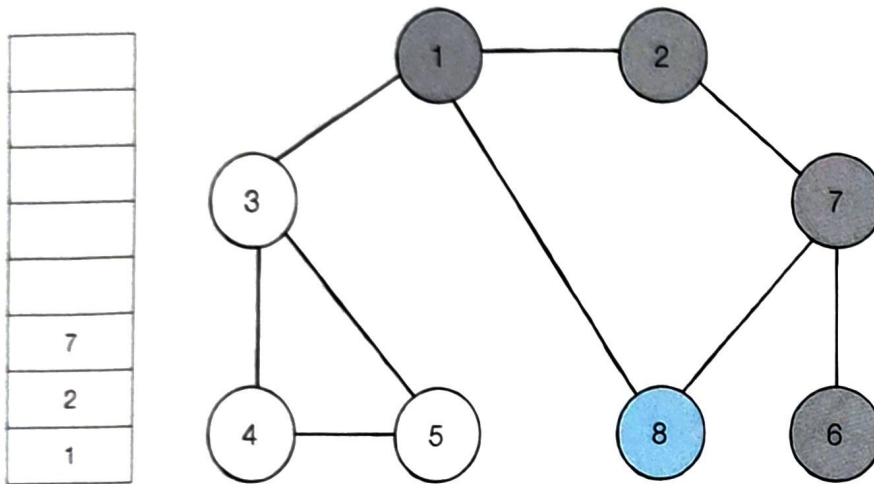
step 5 스택의 최상단 노드인 '6'에 방문하지 않은 인접 노드가 없다. 따라서 스택에서 '6'번 노드를 꺼낸다.



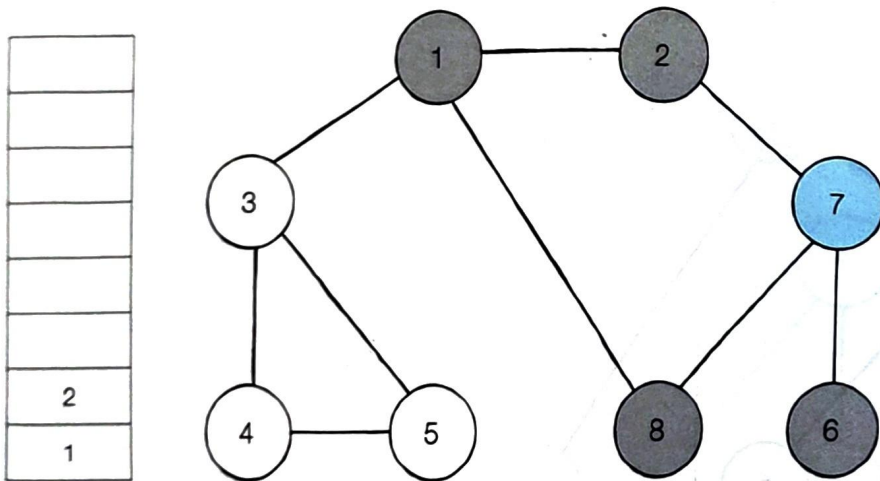
step 6 스택의 최상단 노드인 '7'에 방문하지 않은 인접 노드 '8'이 있다. 따라서 '8'번 노드를 스택에 넣고 방문 처리를 한다.



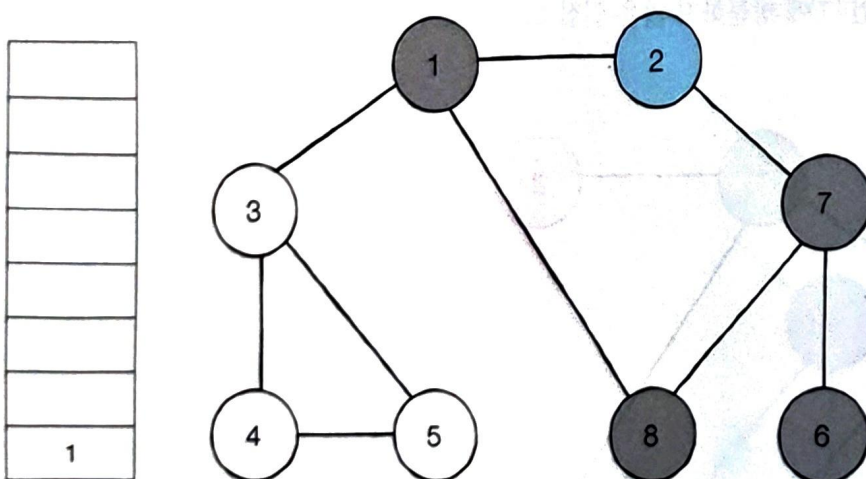
step 7 스택의 최상단 노드인 '8'에 방문하지 않은 인접 노드가 없다. 따라서 스택에서 '8'번 노드를 꺼낸다.



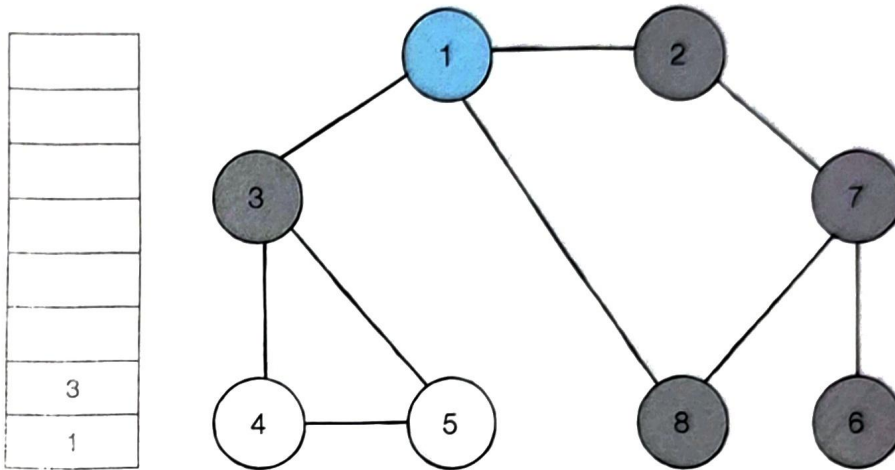
step 8 스택의 최상단 노드인 '7'에 방문하지 않은 인접 노드가 없다. 따라서 스택에서 '7'번 노드를 꺼낸다.



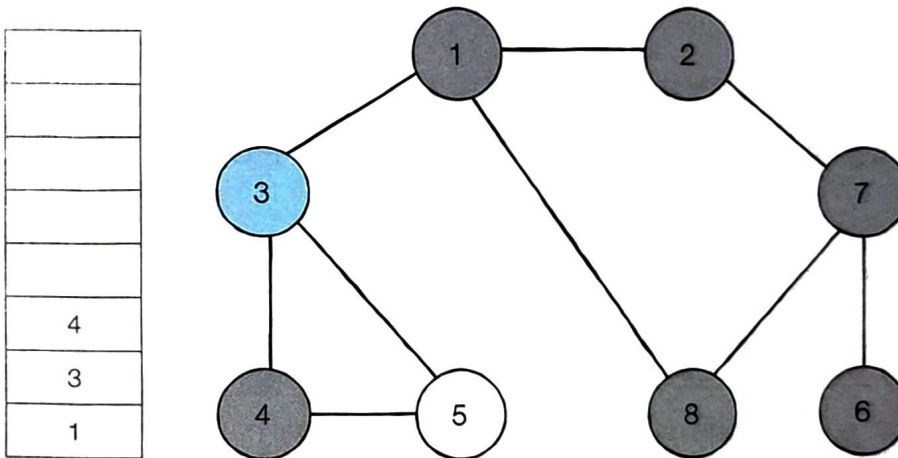
step 9 스택의 최상단 노드인 '2'에 방문하지 않은 인접 노드가 없다. 따라서 스택에서 '2'번 노드를 꺼낸다.



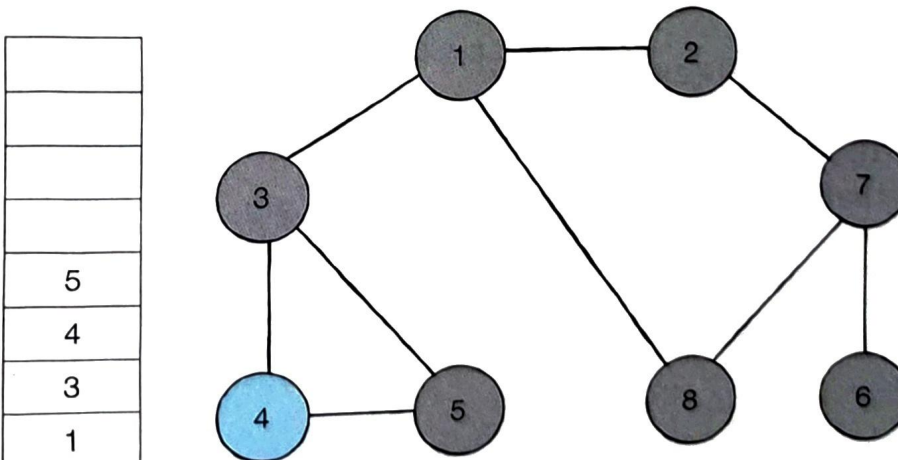
step 10 스택의 최상단 노드인 '1'에 방문하지 않은 인접 노드 '3'을 스택에 넣고 방문 처리한다.



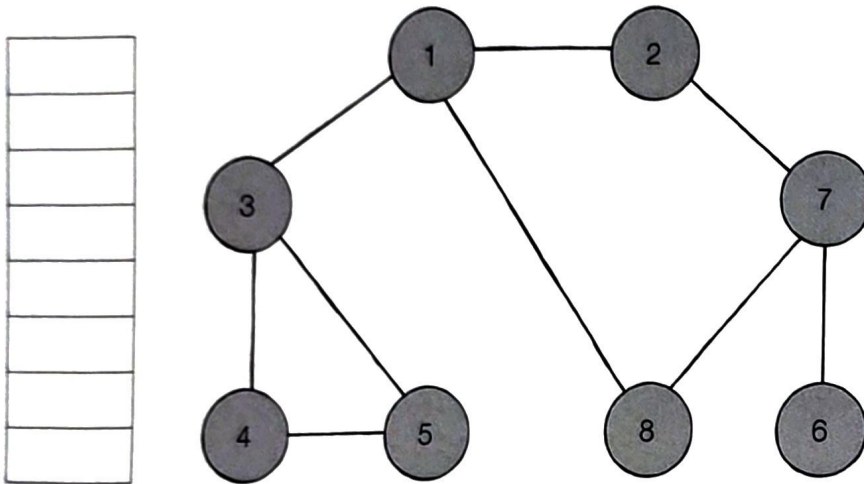
step 11 스택의 최상단 노드인 '3'에 방문하지 않은 인접 노드 '4'와 '5'가 있다. 이 중에서 가장 작은 노드인 '4'를 스택에 넣고 방문 처리를 한다.



step 12 스택의 최상단 노드인 '4'에 방문하지 않은 인접 노드 '5'가 있다. 따라서 '5'번 노드를 스택에 넣고 방문 처리를 한다.



step 13 남아 있는 노드에 방문하지 않은 인접 노드가 없다. 따라서 모든 노드를 차례대로 꺼내면 다음과 같다.



결과적으로 노드의 탐색 순서(스택에 들어간 순서)는 다음과 같다.

1 → 2 → 7 → 6 → 8 → 3 → 4 → 5

깊이 우선 탐색 알고리즘인 DFS는 스택 자료구조에 기초한다는 점에서 구현이 간단하다. 실제로는 스택을 쓰지 않아도 되며 탐색을 수행함에 있어서 데이터의 개수가 N개인 경우 $O(N)$ 의 시간이 소요된다는 특징이 있다.

또한 DFS는 스택을 이용하는 알고리즘이기 때문에 실제 구현은 재귀 함수를 이용했을 때 매우 간결하게 구현할 수 있다. 예제 소스코드는 다음과 같다.

5-8.py DFS 예제

```
# DFS 메서드 정의
def dfs(graph, v, visited):
    # 현재 노드를 방문 처리
    visited[v] = True
    print(v, end=' ')
    # 현재 노드와 연결된 다른 노드를 재귀적으로 방문
    for i in graph[v]:
        if not visited[i]:
            dfs(graph, i, visited)

# 각 노드가 연결된 정보를 리스트 자료형으로 표현(2차원 리스트)
graph = [
    [],
```

```

[2, 3, 8],
[1, 7],
[1, 4, 5],
[3, 5],
[3, 4],
[7],
[2, 6, 8],
[1, 7]
]

```

각 노드가 방문된 정보를 리스트 자료형으로 표현(1차원 리스트)

```
visited = [False] * 9
```

정의된 DFS 함수 호출

```
dfs(graph, 1, visited)
```

```
2 7 6 8 3 4 5
```

BFS

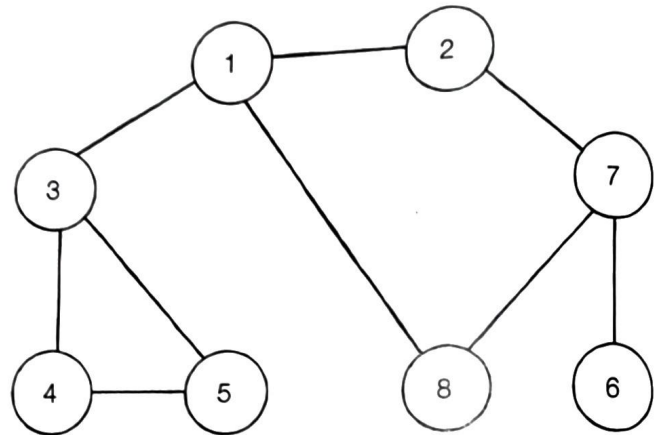
BFSBreadth First Search 알고리즘은 ‘너비 우선 탐색’이라는 의미를 가진다. 쉽게 말해 **가까운 노드부터 탐색하는 알고리즘**이다. DFS는 최대한 멀리 있는 노드를 우선으로 탐색하는 방식으로 동작한다고 했는데, BFS는 그 반대다. 그렇다면 BFS는 실제로 어떤 방식으로 구현할 수 있을까? BFS 구현에서는 선입선출 방식인 큐 자료구조를 이용하는 것이 정석이다. 인접한 노드를 반복적으로 큐에 넣도록 알고리즘을 작성하면 자연스럽게 먼저 들어온 것이 먼저 나가게 되어, 가까운 노드부터 탐색을 진행하게 된다. 그림과 함께 자세한 동작 방식을 살펴보자.

알고리즘의 정확한 동작 방식은 다음과 같다.

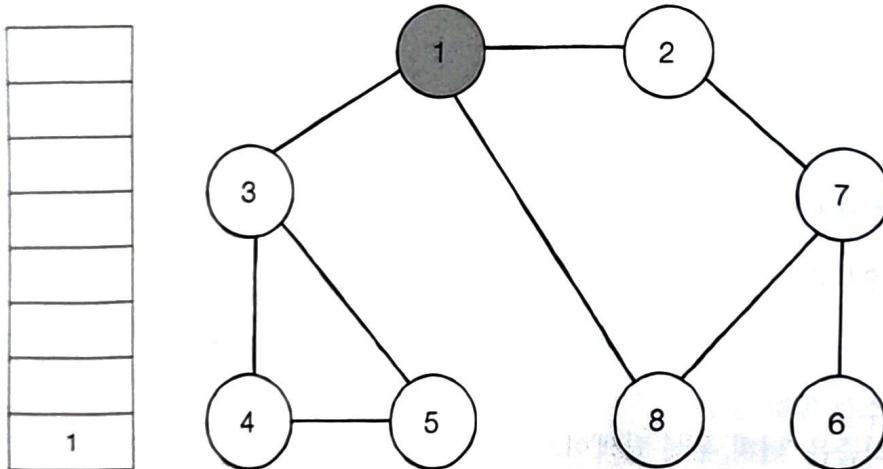
- ❶ 탐색 시작 노드를 큐에 삽입하고 방문 처리를 한다.
- ❷ 큐에서 노드를 꺼내 해당 노드의 인접 노드 중에서 방문하지 않은 노드를 모두 큐에 삽입하고 방문 처리를 한다.
- ❸ ❷번의 과정을 더 이상 수행할 수 없을 때까지 반복한다.

오른쪽과 같은 그래프를 생각해보자.

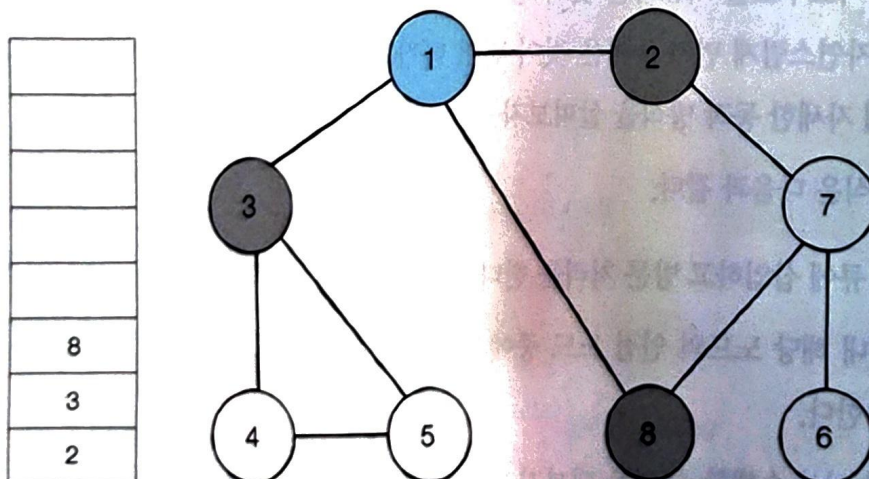
BFS를 이용하여 탐색하면 그 과정은 다음과 같다. 마찬가지로 인접한 노드가 여러 개 있을 때, 숫자가 작은 노드부터 먼저 큐에 삽입한다고 가정한다. 다음 그림에서 큐에 원소가 들어올 때, 위에서 들어오고 아래쪽에서 꺼낸다고 가정하자.



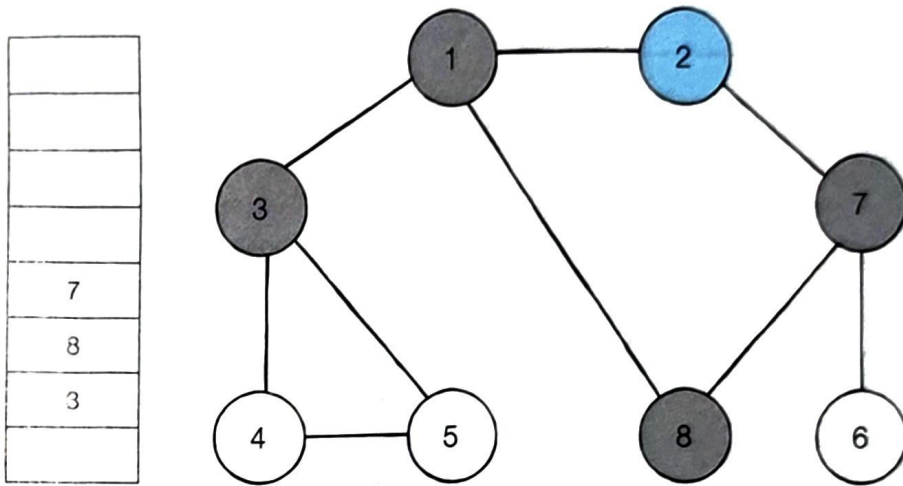
step 1 시작 노드인 '1'을 큐에 삽입하고 방문 처리를 한다. 방문 처리된 노드는 회색으로, 큐에서 꺼내 현재 처리하는 노드는 하늘색으로 표현했다.



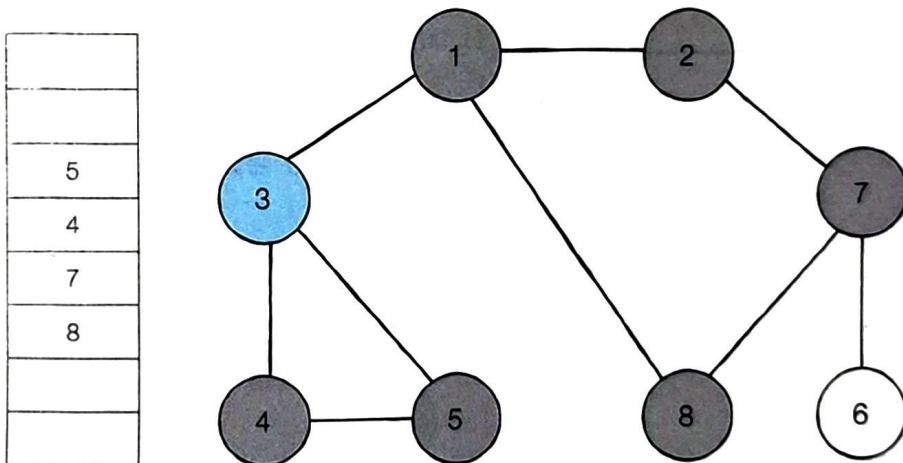
step 2 큐에서 노드 '1'을 꺼내고 방문하지 않은 인접 노드 '2', '3', '8'을 모두 큐에 삽입하고 방문 처리를 한다.



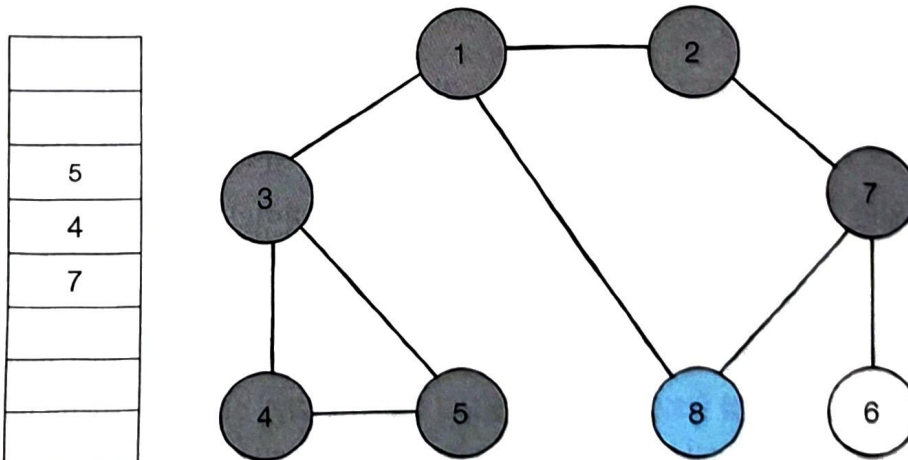
step 3 큐에서 노드 '2'를 꺼내고 방문하지 않은 인접 노드 '7'을 큐에 삽입하고 방문 처리를 한다.



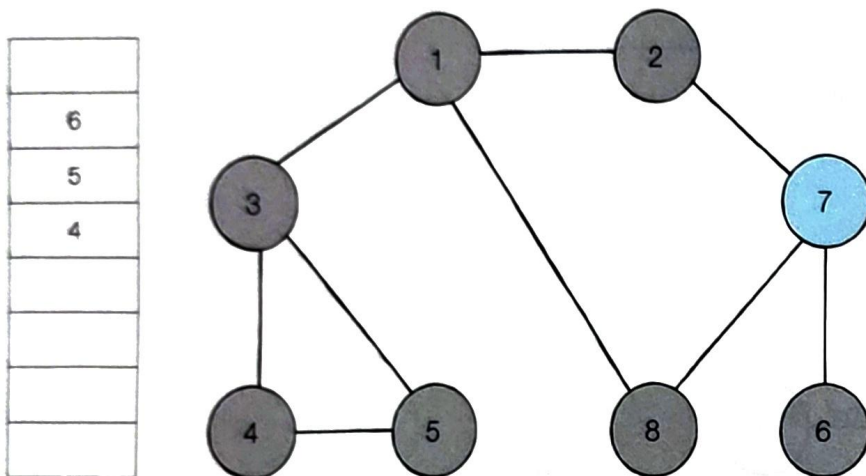
step 4 큐에서 노드 '3'을 꺼내고 방문하지 않은 인접 노드 '4'와 '5'를 모두 큐에 삽입하고 방문 처리를 한다.



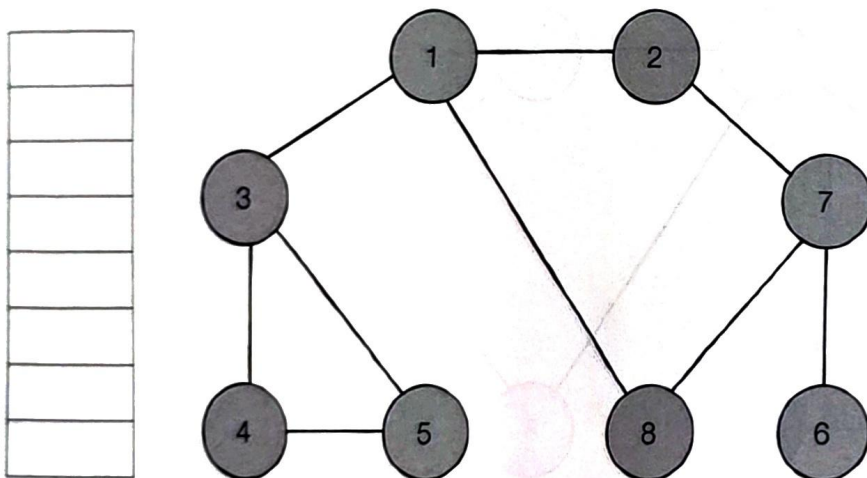
step 5 큐에서 노드 '8'을 꺼내고 방문하지 않은 인접 노드가 없으므로 무시한다.



step 6 큐에서 노드 '7'을 꺼내고 방문하지 않은 인접 노드 '6'을 큐에 삽입하고 방문 처리를 한다.



step 7 남아 있는 노드에 방문하지 않은 인접 노드가 없다. 따라서 모든 노드를 차례대로 꺼내면 최종적으로 다음과 같다.



결과적으로 노드의 탐색 순서(큐에 들어간 순서)는 다음과 같다.

$1 \rightarrow 2 \rightarrow 3 \rightarrow 8 \rightarrow 7 \rightarrow 4 \rightarrow 5 \rightarrow 6$

너비 우선 탐색 알고리즘인 BFS는 큐 자료구조에 기초한다는 점에서 구현이 간단하다. 실제로 구현함에 있어 앞서 언급한 대로 deque 라이브러리를 사용하는 것이 좋으며 탐색을 수행함에 있어 $O(N)$ 의 시간이 소요된다. 일반적인 경우 실제 수행 시간은 DFS보다 좋은 편이라는 점까지만 추가로 기억하자.

TIP 재귀 함수로 DFS를 구현하면 컴퓨터 시스템의 동작 특성상 실제 프로그램의 수행 시간은 느려질 수 있다. 따라서 스택 라이브러리를 이용해 시간 복잡도를 완화하는テクニック이 필요할 때도 있다. 다만, 이 내용은 책의 범위를 벗어나므로, 코딩 테스트에서는 보통 DFS보다는 BFS 구현이 조금 더 빠르게 동작한다는 정도로 기억하자.

5-9.py BFS 예제

```
from collections import deque

# BFS 메서드 정의
def bfs(graph, start, visited):
    # 큐(Queue) 구현을 위해 deque 라이브러리 사용
    queue = deque([start])
    # 현재 노드를 방문 처리
    visited[start] = True
    # 큐가 빌 때까지 반복
    while queue:
        # 큐에서 하나의 원소를 뽑아 출력
        v = queue.popleft()
        print(v, end=' ')
        # 해당 원소와 연결된, 아직 방문하지 않은 원소들을 큐에 삽입
        for i in graph[v]:
            if not visited[i]:
                queue.append(i)
                visited[i] = True

# 각 노드가 연결된 정보를 리스트 자료형으로 표현(2차원 리스트)
graph = [
    [],
    [2, 3, 8],
    [1, 7],
    [1, 4, 5],
    [3, 5],
    [3, 4],
    [7],
    [2, 6, 8],
    [1, 7]
]

# 각 노드가 방문된 정보를 리스트 자료형으로 표현(1차원 리스트)
visited = [False] * 9

# 정의된 BFS 함수 호출
bfs(graph, 1, visited)
```

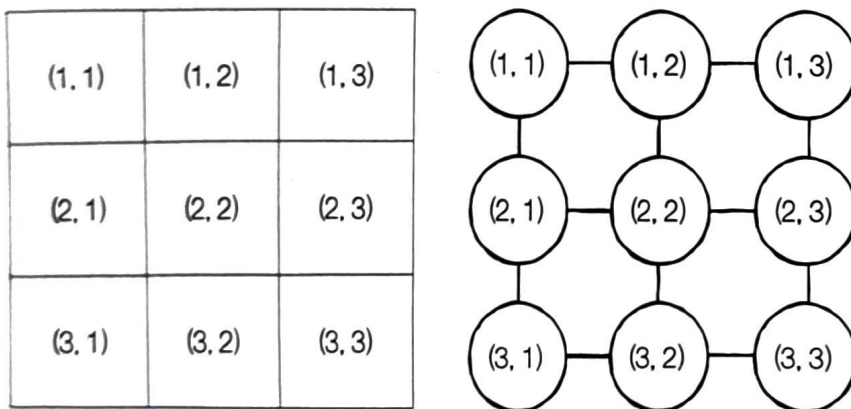
1 2 3 8 7 4 5 6

DFS와 BFS의 구현에 대해 알아보았는데, 간단히 정리하자면 다음 표와 같다. 더 다양한 방식으로 구현할 수 있지만 책에 실은 예제가 가장 간결한 방식이다.

	DFS	BFS
동작 원리	스택	큐
구현 방법	재귀 함수 이용	큐 자료구조 이용

앞서 DFS와 BFS를 설명하는 데 전형적인 그래프 그림을 이용했는데 1차원 배열이나 2차원 배열 또한 그래프 형태로 생각하면 수월하게 문제를 풀 수 있다. 특히나 DFS와 BFS 문제 유형이 그러하다.

예를 들어 게임 맵이 3×3 형태의 2차원 배열이고 각 데이터를 좌표라고 생각해보자. 게임 캐릭터가 (1, 1) 좌표에 있다고 표현할 때처럼 말이다. 이때 각 좌표를 상하좌우로만 이동할 수 있다면 어떨까? 모든 좌표의 형태를 다음처럼 그래프의 형태로 바꿔서 생각할 수 있다.



코딩 테스트 중 2차원 배열에서의 탐색 문제를 만나면 이렇게 그래프 형태로 바꿔서 생각하면 풀이 방법을 조금 더 쉽게 떠올릴 수 있다. 그러므로 코딩 테스트에서 탐색 문제를 보면 그래프 형태로 표현한 다음 풀이법을 고민하도록 하자. 이제 지금까지 배운 내용을 토대로 실전 문제를 풀어보자.



실전 문제

음료수 얼려 먹기

난이도 ●○○ | 풀이 시간 30분 | 시간 제한 1초 | 메모리 제한 128MB

$N \times M$ 크기의 얼음 틀이 있다. 구멍이 뚫려 있는 부분은 0, 칸막이가 존재하는 부분은 1로 표시된다. 구멍이 뚫려 있는 부분끼리 상, 하, 좌, 우로 붙어 있는 경우 서로 연결되어 있는 것으로 간주한다. 이때 얼음 틀의 모양이 주어졌을 때 생성되는 총 아이스크림의 개수를 구하는 프로그램을 작성하시오. 다음의 4×5 얼음 틀 예시에서는 아이스크림이 총 3개 생성된다.

```
00110
00011
11111
00000
```

0	0	1	1	0
0	0	0	1	1
1	1	1	1	1
0	0	0	0	0

입력 조건

- 첫 번째 줄에 얼음 틀의 세로 길이 N 과 가로 길이 M 이 주어진다. ($1 \leq N, M \leq 1,000$)
- 두 번째 줄부터 $N + 1$ 번째 줄까지 얼음 틀의 형태가 주어진다.
- 이때 구멍이 뚫려있는 부분은 0, 그렇지 않은 부분은 1이다.

출력 조건

- 한 번에 만들 수 있는 아이스크림의 개수를 출력한다.

입력 예시

```

15 14
00000111100000
1111101111110
1101101101110
1101101100000
1101111111111
1101111111100
1100000001111
0111111111111
0000000001111
0111111111000
0001111111000
00000001111000
1111111110011
1110001111111
1110001111111
    
```

출력 예시

8

문제 해설

이 문제는 DFS로 해결할 수 있다. 일단 앞에서 배운 대로 얼음을 얼릴 수 있는 공간이 상, 하, 좌, 우로 연결되어 있다고 표현할 수 있으므로 그래프 형태로 모델링 할 수 있다. 예를 들어 다음과 같이 3×3 크기의 얼음 틀이 있다고 가정하자.

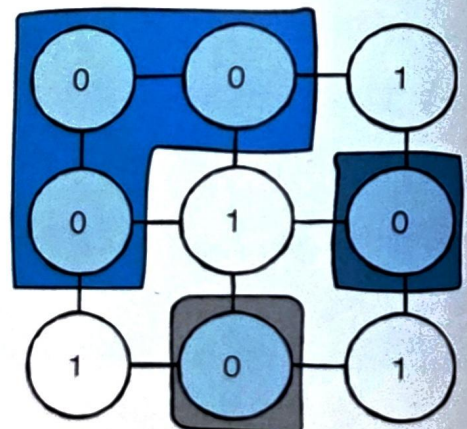
```

001
010
101
    
```

이는 오른쪽과 같이 그래프로 모델링 할 수 있다.

'0'인 값이 상, 하, 좌, 우로 연결되어 있는 노드를 묶으면 오른쪽과 같이 세 묶음이 나올 것이다. 이러한 묶음을 찾아주는 프로그램은 어떻게 작성할 수 있을까?

DFS를 이용하면 간단히 해결할 수 있다.



- 1 특정한 지점의 주변 상, 하, 좌, 우를 살펴본 뒤에 주변 지점 중에서 값이 '0'이면서 아직 방문하지 않은 지점이 있다면 해당 지점을 방문한다.
- 2 방문한 지점에서 다시 상, 하, 좌, 우를 살펴보면서 방문을 다시 진행하면, 연결된 모든 지점을 방문할 수 있다.
- 3 1 ~ 2번의 과정을 모든 노드에 반복하며 방문하지 않은 지점의 수를 센다.

소스코드 예시는 다음과 같다.

5-10.py 답안 예시

```
# N, M을 공백으로 구분하여 입력받기
n, m = map(int, input().split())

# 2차원 리스트의 맵 정보 입력받기
graph = []
for i in range(n):
    graph.append(list(map(int, input())))

# DFS로 특정한 노드를 방문한 뒤에 연결된 모든 노드들도 방문
def dfs(x, y):
    # 주어진 범위를 벗어나는 경우에는 즉시 종료
    if x <= -1 or x >= n or y <= -1 or y >= m:
        return False
    # 현재 노드를 아직 방문하지 않았다면
    if graph[x][y] == 0:
        # 해당 노드 방문 처리
        graph[x][y] = 1
        # 상, 하, 좌, 우의 위치도 모두 재귀적으로 호출
        dfs(x - 1, y)
        dfs(x, y - 1)
        dfs(x + 1, y)
        dfs(x, y + 1)
        return True
    return False

# 모든 노드(위치)에 대하여 음료수 채우기
result = 0
for i in range(n):
    for j in range(m):
        # 현재 위치에서 DFS 수행
        if dfs(i, j) == True:
            result += 1

print(result) # 정답 출력
```



실전 문제

미로 탈출

난이도 ●○○ | 풀이 시간 30분 | 시간 제한 1초 | 메모리 제한 128MB

동빈이는 $N \times M$ 크기의 직사각형 형태의 미로에 갇혀 있다. 미로에는 여러 마리의 괴물이 있어 이를 피해 탈출해야 한다. 동빈이의 위치는 (1, 1)이고 미로의 출구는 (N, M)의 위치에 존재하며 한 번에 한 칸씩 이동할 수 있다. 이때 괴물이 있는 부분은 0으로, 괴물이 없는 부분은 1로 표시되어 있다. 미로는 반드시 탈출할 수 있는 형태로 제시된다. 이때 동빈이가 탈출하기 위해 움직여야 하는 최소 칸의 개수를 구하시오. 칸을 셀 때는 시작 칸과 마지막 칸을 모두 포함해서 계산한다.

입력 조건 • 첫째 줄에 두 정수 N, M ($4 \leq N, M \leq 200$)이 주어집니다. 다음 N 개의 줄에는 각각 M 개의 정수(0 혹은 1)로 미로의 정보가 주어진다. 각각의 수들은 공백 없이 붙어서 입력으로 제시된다. 또한 시작 칸과 마지막 칸은 항상 1이다.

출력 조건 • 첫째 줄에 최소 이동 칸의 개수를 출력한다.

입력 예시

```
5 6
101010
111111
000001
111111
111111
```

출력 예시

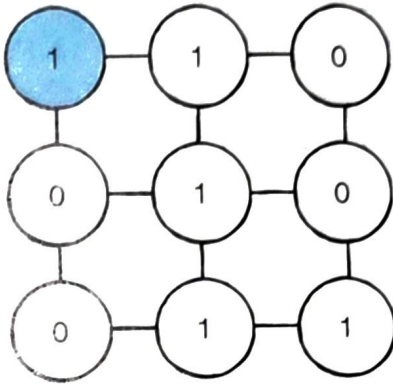
10

문제 해설

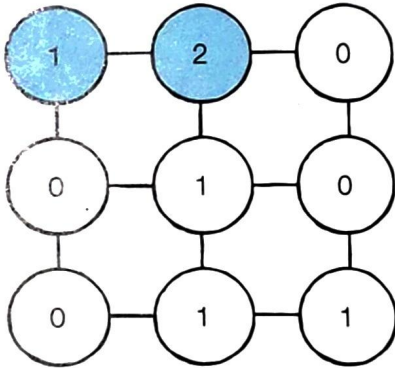
이 문제는 BFS를 이용했을 때 매우 효과적으로 해결할 수 있다. BFS는 시작 지점에서 가까운 노드부터 차례대로 그래프의 모든 노드를 탐색하기 때문이다. 그러므로 (1, 1) 지점에서부터 BFS를 수행하여 모든 노드의 값을 거리 정보로 넣으면 된다. 특정한 노드를 방문하면 그 이전 노드의 거리에 1을 더한 값을 리스트에 넣는다. 예를 들어 만약 미로의 크기가 3×3 이며 오른쪽과 같이 구성되어 있다고 가정해보자.

```
110
010
011
```

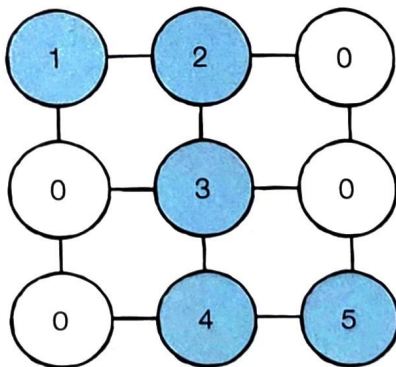
step 1 맨 처음에 (1, 1)의 위치에서 시작하며, (1, 1)의 값은 항상 1이라고 문제에서 언급되어 있다.



step 2 (1, 1) 좌표에서 상, 하, 좌, 우로 탐색을 진행하면 바로 옆 노드인 (1, 2) 위치의 노드를 방문하게 되고 새롭게 방문하는 (1, 2) 노드의 값을 2로 바꾸게 된다.



step 3 마찬가지로 BFS를 계속 수행하면 결과적으로 다음과 같이 최단 경로의 값들이 1씩 증가하는 형태로 변경된다.



참고로 소스코드 상에서, 첫 번째 시작 위치는 다시 방문할 수 있도록 되어 첫 번째 시작 위치에 해당하는 값이 3으로 변경될 여지가 있다. 하지만 본 문제에서는 단순히 가장 오른쪽 아래 위치로 이동하는 것을 요구하고 있기 때문에, 본 소스코드는 정상적으로 답을 도출하는 간결한 정답 코드이다.

5-11.py 답안 예시

```
from collections import deque

# N, M을 공백으로 구분하여 입력받기
n, m = map(int, input().split())
# 2차원 리스트의 맵 정보 입력받기
graph = []
for i in range(n):
    graph.append(list(map(int, input()))))

# 이동할 네 방향 정의(상, 하, 좌, 우)
dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]

# BFS 소스코드 구현
def bfs(x, y):
    # 큐(Queue) 구현을 위해 deque 라이브러리 사용
    queue = deque()
    queue.append((x, y))
    # 큐가 빌 때까지 반복
    while queue:
        x, y = queue.popleft()
        # 현재 위치에서 네 방향으로의 위치 확인
        for i in range(4):
            nx = x + dx[i]
            ny = y + dy[i]
            # 미로 찾기 공간을 벗어난 경우 무시
            if nx < 0 or ny < 0 or nx >= n or ny >= m:
                continue
            # 벽인 경우 무시
            if graph[nx][ny] == 0:
                continue
            # 해당 노드를 처음 방문하는 경우에만 최단 거리 기록
            if graph[nx][ny] == 1:
                graph[nx][ny] = graph[x][y] + 1
                queue.append((nx, ny))
    # 가장 오른쪽 아래까지의 최단 거리 반환
    return graph[n - 1][m - 1]

# BFS를 수행한 결과 출력
print(bfs(0, 0))
```

CHAPTER



06

정렬

연속된 데이터를 기준에 따라서 정렬하기 위한 알고리즘